```
            return None

        decoded = self._decode_compressed_vrs(frame_bytes)
        if decoded is None:
            # Legacy fallback when frame is not Compressed VRS.
            icao = frame_bytes[4:7].hex().upper() if len(frame_bytes) >= 7 else "UNKNOWN"
            callsign = self._decode_callsign(frame_bytes)
            if callsign_hint:
                callsign = callsign_hint.strip().upper()

            prev = self._state.get(icao if icao and icao != "000000" else callsign, {})
            prev_lat = prev.get("latDD")
            prev_lon = prev.get("lonDD")
            lat, lon = self._decode_lat_lon(frame_bytes, prev_lat=prev_lat, prev_lon=prev_lon)

            decoded = {
                "icaoAddress": icao,
                "callsign": callsign,
            }
            if lat is not None:
                decoded["latDD"] = lat
                decoded["lonDD"] = lon
            if len(frame_bytes) >= 21:
                altitude_ft = int.from_bytes(frame_bytes[19:21], "big")
                if altitude_ft > 0:
                    decoded["altitudeMM"] = int(round(altitude_ft * 304.8))

        if callsign_hint:
            decoded["callsign"] = callsign_hint.strip().upper()

        icao = decoded.get("icaoAddress", "UNKNOWN")
        callsign = decoded.get("callsign", "UNKNOWN")
        record_key = icao if icao and icao != "000000" else callsign
        if not record_key:
            return None

        now_epoch = time.time()
        iso_time = timestamp or _now_iso()

        prev = self._state.get(record_key, {})
        merged = {
            "icaoAddress": icao or prev.get("icaoAddress", "UNKNOWN"),
            "callsign": callsign or prev.get("callsign", "UNKNOWN"),
            "latDD": prev.get("latDD", 0.0),
            "lonDD": prev.get("lonDD", 0.0),
            "altitudeMM": prev.get("altitudeMM", 0),
            "headingDE2": prev.get("headingDE2", 0),
            "horVelocityCMS": prev.get("horVelocityCMS", 0),
            "timestamp": iso_time,
            "frameLength": len(frame_bytes),
            "sourceIp": source_ip or prev.get("sourceIp", ""),
            "_last_seen": now_epoch,
        }

        for key in (
            "latDD",
            "lonDD",
            "altitudeMM",
            "headingDE2",
            "horVelocityCMS",
            "verVelocityCMS",
            "squawk",
            "compressedFlags",
        ):
            if key in decoded and decoded[key] is not None:
                if key in ("headingDE2", "horVelocityCMS", "verVelocityCMS", "squawk"):
                    new_value = decoded[key]
                    old_value = merged.get(key, 0)
                    if new_value == 0 and old_value not in (None, 0):
                        continue
                merged[key] = decoded[key]

        self._state[record_key] = merged
        return dict(merged)
```